Vehicle Management System

A
Project Report

Full stack developer

Bachelor of Computer Science and Engineering

Submitted by

AP23110010404 – T. Ramya Sri
AP23110010407 – Ch. Rahitya
AP23110010410 – S. Sowmya Sri

Submitted to

Mr. Himanshu Mishra

SRM UNIVERSITY-AP
MANGALAGIRI(M)
KURAGALLU VILLAGE
ANDHRA PRADESH – 522540

INTRODUCTION:

The **Vehicle Service Management Platform** is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). It is designed to simplify and digitalize vehicle service operations by connecting customers, administrators, and technicians on a single platform.

The system allows users to register, manage their vehicles, book service appointments, and track service history. It provides a structured workflow where administrators manage bookings and assign technicians, while technicians update service progress and maintain records.

The platform ensures secure authentication, efficient data handling, and a user-friendly interface, making it a reliable solution for modern vehicle service management.

SCENARIO BASED CASE STUDY:

Meet Ramya, a working professional who owns multiple vehicles and often struggles to manage their servicing schedules.
She discovers the Vehicle Service Management Platform, which helps her streamline her vehicle maintenance.

- User Registration & Login:
  Ramya creates an account and securely logs in using her credentials.
- Vehicle Management:
  She adds her vehicles (e.g., Activa, Maruthi Suzuki) with details like license plate and type.
- Service Booking:
  She books a service by selecting:
  - Vehicle
  - Service type (Oil Change, Brake Repair, etc.)
  - Preferred date and time
- Admin Interaction:
  The admin receives her request and assigns a technician.
- Technician Workflow:
  The technician performs the service and updates:
  - Repairs performed

    - Parts replaced
    - Notes
- Service History:
  Ramya can view complete service history anytime.

This platform helps her save time, stay organized, and maintain her vehicles efficiently.

SYSTEM REQUIREMENTS:
Software Requirements:

To ensure the smooth development, deployment, and execution of the **Vehicle Service Management**, certain system requirements must be satisfied. These requirements are categorized into software requirements and hardware requirements, which together ensure efficient performance, scalability, and reliability of the application.

1. **Software Requirements:**

The following software components are essential for building, testing, and running the application successfully:

a. Operating System:

The application supports cross-platform development and execution. Any of the following operating systems can be used:

- Windows 10 / Windows 11
- macOS
- Linux (Ubuntu or other distributions)

These operating systems provide the necessary environment for installing development tools and running both frontend and backend services.

b. Node.js (Version 16 or above):

Node.js is used as the runtime environment for executing JavaScript on the server side. It plays a crucial role in:

- Running backend services
- Handling API requests and responses
- Managing asynchronous operations efficiently

A stable version (v16 or higher) is recommended for better performance and compatibility.

c. npm (Node Package Manager):

npm is used for managing project dependencies. It allows developers to:

- Install required libraries and frameworks
- Manage versions of packages
- Run scripts for development and production

It is automatically installed with Node.js.

d. React.js:

React.js is a frontend JavaScript library used to build dynamic and responsive user interfaces. It is responsible for:

- Creating reusable UI components
- Managing application state
- Rendering views efficiently

React enables a smooth user experience through fast updates and component-based architecture.

e. Vite (Build Tool):

Vite is used as a modern frontend build tool that provides:

- Faster development server startup
- Hot module replacement (HMR)
- Optimized production builds

It significantly improves development speed compared to traditional tools.

f. Express.js:

Express.js is a lightweight web framework for Node.js used to:

- Build RESTful APIs
- Handle routing and middleware
- Manage HTTP requests and responses

It forms the backbone of the backend application.

g. MongoDB:

MongoDB is a NoSQL database used to store application data such as:

- User details
- Vehicle information
- Appointments
- Service records

It provides flexibility in handling unstructured and semi-structured data.

h. Mongoose:

Mongoose is an Object Data Modeling (ODM) library for MongoDB. It helps in:

- Defining schemas and models
- Validating data
- Performing CRUD operations

It simplifies interaction between Node.js and MongoDB.

i. Postman:

Postman is used for testing APIs during development. It allows developers to:

- Send HTTP requests (GET, POST, PUT, DELETE)
- Test backend endpoints
- Debug API responses

j. Visual Studio Code (VS Code):

VS Code is the preferred code editor for development. It provides:

- Syntax highlighting
- Extensions support
- Integrated terminal
- Debugging tools

k. Web Browser:

A modern web browser is required to run and test the frontend application:

- Google Chrome (recommended)
- Mozilla Firefox

Browsers help in rendering the UI and testing responsiveness.

l. Additional Libraries & Tools:

The project also uses several supporting libraries:

- Axios → For API communication
- React Router → For navigation
- Tailwind CSS → For styling
- JWT → For authentication
- Bcrypt → For password encryption

2. Hardware Requirements:

The following hardware specifications are recommended to ensure smooth development and execution:

a. Processor:

- Minimum: Intel Core i5 (8th Generation) or AMD Ryzen 5
- Recommended: Intel i7 or higher

A powerful processor ensures faster compilation, efficient multitasking, and smooth running of both frontend and backend servers.

b. RAM (Memory):

- Minimum: 8 GB
- Recommended: 16 GB

Adequate RAM is necessary to:

- Run development servers
- Use IDEs like VS Code
- Open multiple browser tabs for testing

c. Storage:

- Minimum: 1 GB free space
- Recommended: SSD storage

Storage is required for:

- Project files
- Node modules
- Database setup

SSD improves performance significantly compared to HDD.

d. Display

- Minimum resolution: 1366 × 768
- Recommended: Full HD (1920 × 1080)

A higher resolution display provides better visibility for coding and UI design.

e. Internet Connectivity

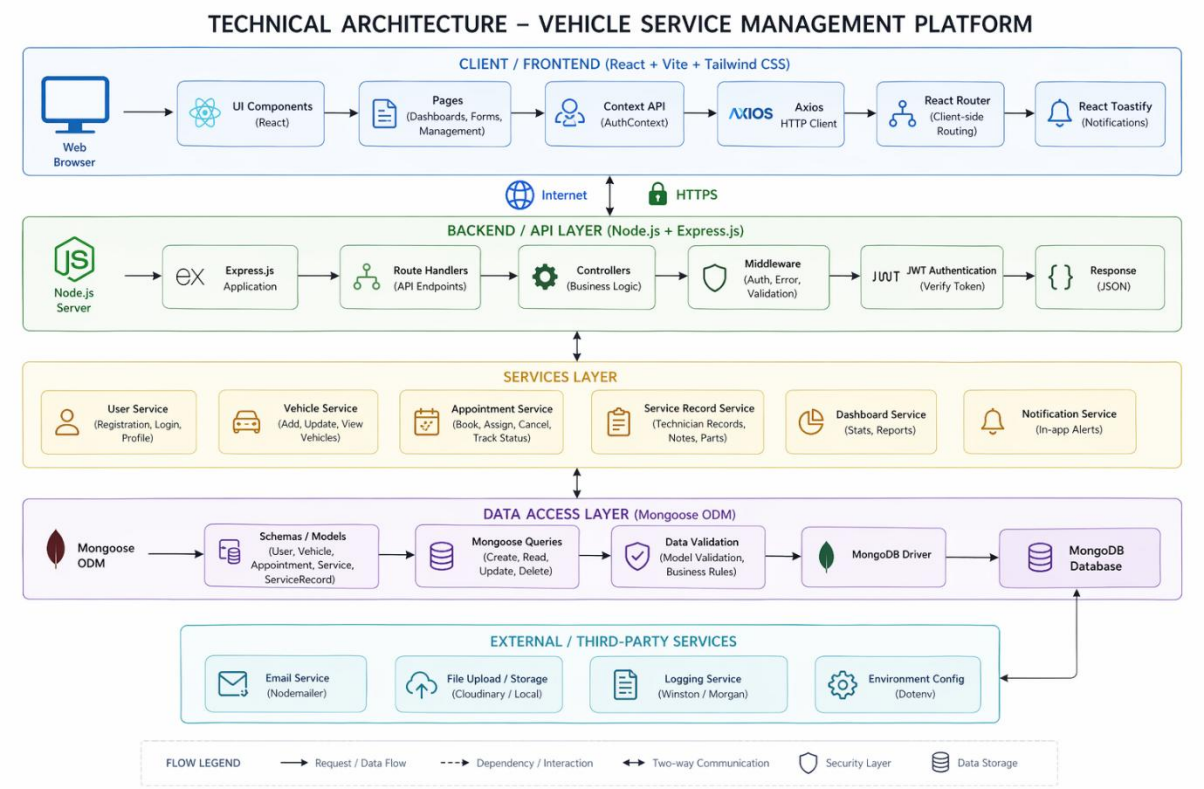
A stable internet connection is required for:

- Installing dependencies (npm packages)
- Accessing APIs

- Using cloud services (if applicable)

PROJECT ARCHITECTURE:

TECHNICAL ARCHITECTURE:



The Vehicle Service Management Platform is built using the MERN stack and follows a layered architecture that separates the system into frontend, backend, services, data access, and external service layers. This structure ensures better scalability, maintainability, and security of the application.

The frontend layer is developed using React.js, Vite, and Tailwind CSS, and is responsible for handling user interaction and displaying the interface. It consists of reusable UI components such as Navbar, Sidebar, and forms, along with pages like Login, Register, Dashboard, Vehicle Management, Booking, and Service History. React Router is used for navigation without page reloads, while the Context API manages global state such as authentication and user roles. Axios is used to send HTTP requests to the backend, and React Toastify is used to display notifications for user actions.

Communication between the frontend and backend occurs over the internet using secure HTTPS protocols, ensuring safe data transfer. The frontend sends requests to backend APIs, and the backend responds with JSON data, which is used to update the UI dynamically.

The backend layer is built using Node.js and Express.js. Node.js provides the runtime environment, while Express.js is used to create RESTful APIs and manage routing. The backend includes route handlers that define API endpoints and controllers that implement the business logic. Middleware is used for authentication, validation, and error handling. JWT (JSON Web Tokens) is used for secure authentication, allowing role-based access control for Customers, Admins, and Technicians.

The services layer organizes the core functionalities of the system, including user service (registration and login), vehicle service (managing vehicles), appointment service (booking and tracking), service record service (technician updates), dashboard service (statistics), and notification service. This modular design improves code organization and reusability.

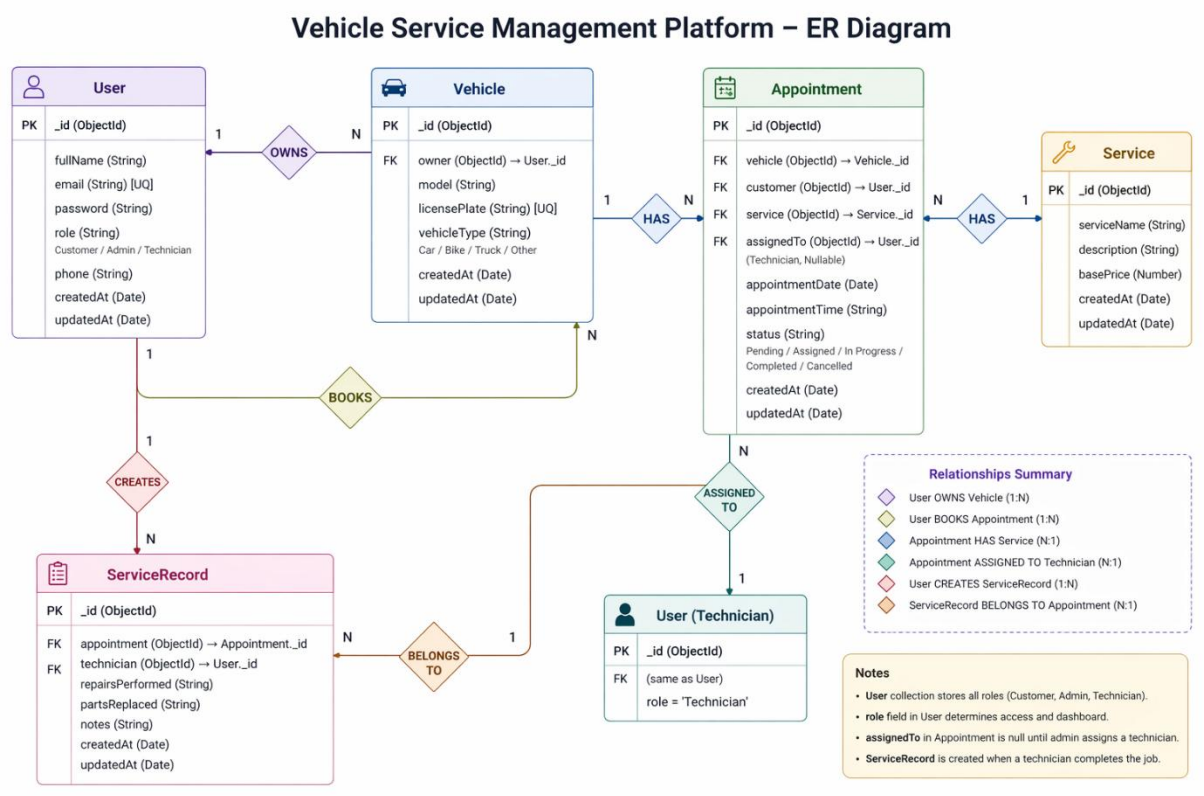
The data access layer uses MongoDB as the database and Mongoose as the ODM to manage data. It defines schemas for entities such as User, Vehicle, Appointment, and ServiceRecord. Mongoose handles CRUD operations and ensures data validation before storing it in the database.

The system also integrates external services such as email notifications, file storage, logging tools, and environment configuration using dotenv for managing sensitive data securely.

The overall data flow starts when the user interacts with the frontend. The request is sent to the backend via Axios, processed through middleware and controllers, and data is stored or retrieved from the database. The backend then sends a JSON response, which updates the frontend interface.

This architecture provides advantages such as scalability, modular design, security through authentication and validation, and efficient performance, making the system reliable and easy to maintain.

ER DIAGRAM:



The ER diagram of the **Vehicle Service Management Platform** represents the structure of the database and the relationships between different entities in the system. The main entities involved are **User, Vehicle, Appointment, Service, and ServiceRecord**, which together support the complete workflow of vehicle servicing.

The **User** entity stores details of all users in the system, including customers, admins, and technicians. It contains attributes such as user ID, full name, email, password, role, phone number, and timestamps. The role attribute determines whether the user is a customer, admin, or technician, and controls access to different functionalities in the system.

The **Vehicle** entity stores information about vehicles registered by users. Each vehicle has attributes such as vehicle ID, model, license plate, type, and timestamps. There is a **one-to-many relationship between User and Vehicle**, meaning one user can own multiple vehicles, but each vehicle belongs to only one user.

The **Appointment** entity represents service bookings made by customers. It includes attributes such as appointment ID, vehicle reference, customer reference, service type, assigned technician, date, time, and status. The status field indicates the current stage of the appointment, such as pending, assigned, in progress, completed, or cancelled. There is a **one-to-many relationship between Vehicle and Appointment**, meaning one vehicle can have multiple service appointments.

The **Service** entity defines the types of services offered, such as oil change, brake repair, and general maintenance. It includes attributes like service ID, service name, description, and base price. Each appointment is associated with one service, creating a **many-to-one relationship between Appointment and Service**.

The **ServiceRecord** entity stores detailed information about completed services. It includes attributes such as service record ID, appointment reference, technician reference, repairs performed, parts replaced, notes, and timestamps. This entity is created when a technician completes a service. There is a **one-to-many relationship between Appointment and ServiceRecord**, where each appointment results in one service record.

Additionally, the system includes relationships such as **User books Appointment**, where a customer creates service bookings, and **Appointment assigned to Technician**, where an admin assigns a technician to handle the service. The technician is also linked to the service record, as they are responsible for updating service details.

Overall, the ER diagram clearly defines how data flows between users, vehicles, appointments, and service records. It ensures proper data organization, avoids redundancy, and supports efficient management of vehicle servicing operations within the system.

Key Features:

The Vehicle Service Management Platform provides secure user authentication using JWT and password encryption, along with role-based access for customers, admins, and technicians. Each user has a dedicated dashboard where customers manage vehicles and bookings, admins assign technicians and monitor operations, and technicians handle assigned tasks.

The system allows users to add and manage multiple vehicles and book service appointments by selecting service type, date, and time. Admins assign technicians to these appointments, and the system tracks progress through stages like pending, assigned, in progress, and completed.

Technicians update service records after completing tasks, including repairs and notes, which are stored as service history for future reference. Customers can view this history to track vehicle maintenance. A cancellation feature is also available, but only before a technician is assigned.

The platform includes notifications for user actions and ensures a responsive interface. It also maintains secure data handling through validation, protected APIs, and HTTPS communication, making the system efficient and reliable.

ROLES AND RESPONSIBILITIES:

User (Customer):-

- **Registration and Account Management:** Customers are responsible for creating an account, providing accurate personal details, and managing their profile information, including updating details and maintaining password security.

- **Vehicle Management:** Customers can add, update, and manage their vehicles by entering details such as model, license plate, and vehicle type. They are responsible for keeping this information accurate.

- **Service Booking:** Customers can book service appointments by selecting a vehicle, service type, date, and time. They must ensure correct details are provided during booking.

- **Appointment Tracking:** Customers can track the status of their bookings (Pending, Assigned, In Progress, Completed, or Cancelled) and stay updated on service progress.

- **Booking Cancellation:** Customers can cancel appointments only if they are still pending and not assigned to a technician.

- **Service History Access:** Customers can view past service records, including repairs, parts replaced, and technician notes, to monitor vehicle maintenance.

- **Communication:** Customers may contact support or service providers for queries, changes, or issues related to bookings.

Admin:-

- **System Management:** The admin is responsible for managing overall system operations, monitoring performance, and ensuring smooth functioning of the platform.
- **User Management:** The admin manages user accounts, including verification, role assignment, and handling any issues related to users.
- **Appointment Management:** The admin oversees all service bookings and ensures proper scheduling and workflow.
- **Technician Assignment:** The admin assigns technicians to appointments based on availability and workload.
- **Service Monitoring:** The admin tracks the status of all appointments and ensures services are completed efficiently.

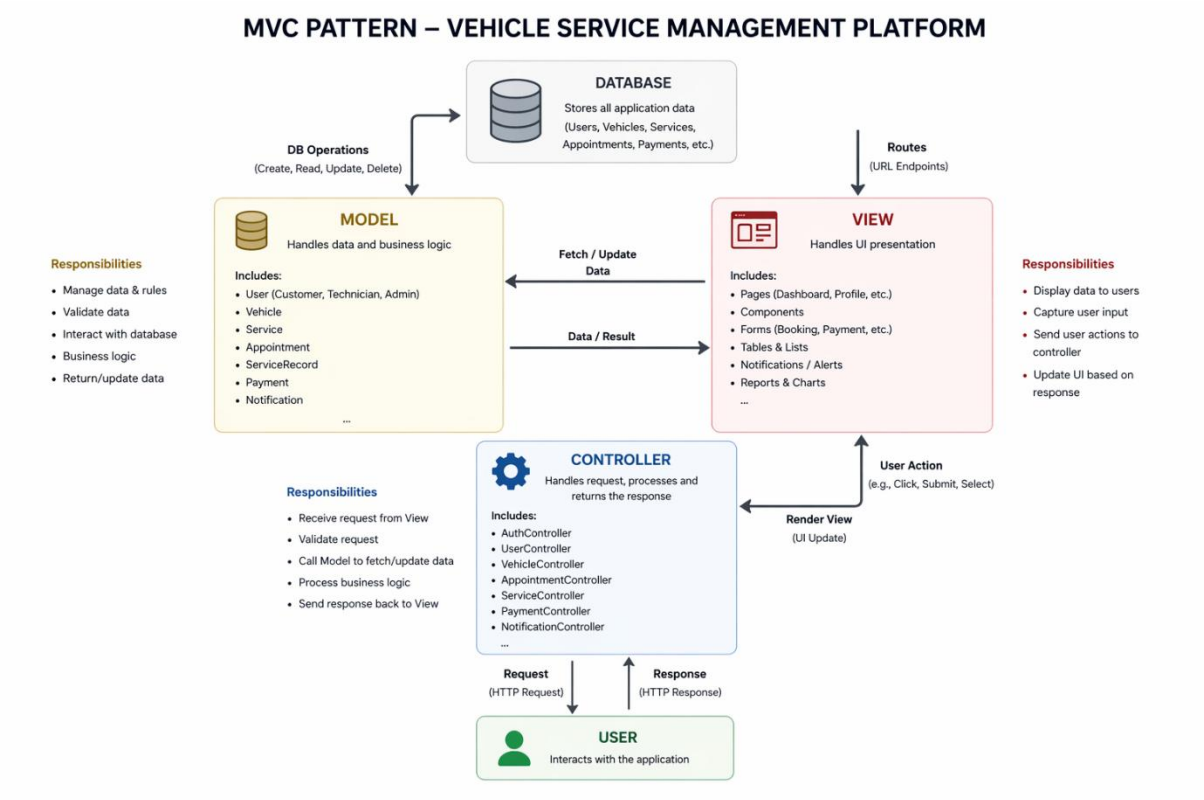
Technician

- **Task Management:** Technicians can view all service tasks assigned to them by the admin.
- **Service Execution:** Technicians are responsible for performing the required vehicle service based on the assigned appointment.
- **Progress Updates:** Technicians update the status of tasks (In Progress → Completed) to reflect real-time progress.
- **Service Record Creation:** After completing a service, technicians must record details such as repairs performed, parts replaced, and additional notes.

- **Quality Assurance:** Technicians ensure that services are performed correctly and meet required standards.

User Flow:

MVC PATTERN:



The Vehicle Service Management Platform follows the **Model-View-Controller (MVC)** architectural pattern, which divides the application into three interconnected layers: Model, View, and Controller. This separation helps in organizing the code efficiently, improving maintainability, scalability, and reusability of the system.

Model Layer (Data Layer):

The Model layer is responsible for handling all data-related operations and business logic of the application. It defines the structure of the data using schemas and interacts directly with the database. In this project, the models are implemented using **Mongoose**, which provides a schema-based solution for working with MongoDB.

The Model layer includes entities such as **User, Vehicle, Service, Appointment, and ServiceRecord**. It performs database operations like create, read, update, and delete (CRUD), validates data, enforces business rules, and ensures data consistency. This layer acts as a bridge between the application and the database, managing how data is stored and retrieved.

Controller Layer:
The Controller layer acts as an intermediary between the View and the Model. It handles incoming requests from the client, processes the input, and returns appropriate responses. When a user performs an action (such as booking a service or logging in), the request is sent to the controller through API routes.

The controller validates the request, applies necessary business logic, and interacts with the Model to fetch or update data. After processing, it sends a response back to the frontend in JSON format. In this project, controllers include modules like **AuthController, UserController, VehicleController, AppointmentController, and ServiceController**.

View Layer (Routing / Frontend Layer):
The View layer is responsible for presenting data to the user and capturing user input. In this project, the View is implemented using **React.js** for the frontend and **Express routes** for handling API endpoints.

The frontend consists of pages such as dashboards, login, registration, vehicle management, booking forms, and service history. These components display data received from the backend and allow users to interact with the system. The routing layer defines endpoints (GET, POST, PUT, DELETE) and directs requests to the appropriate controllers.

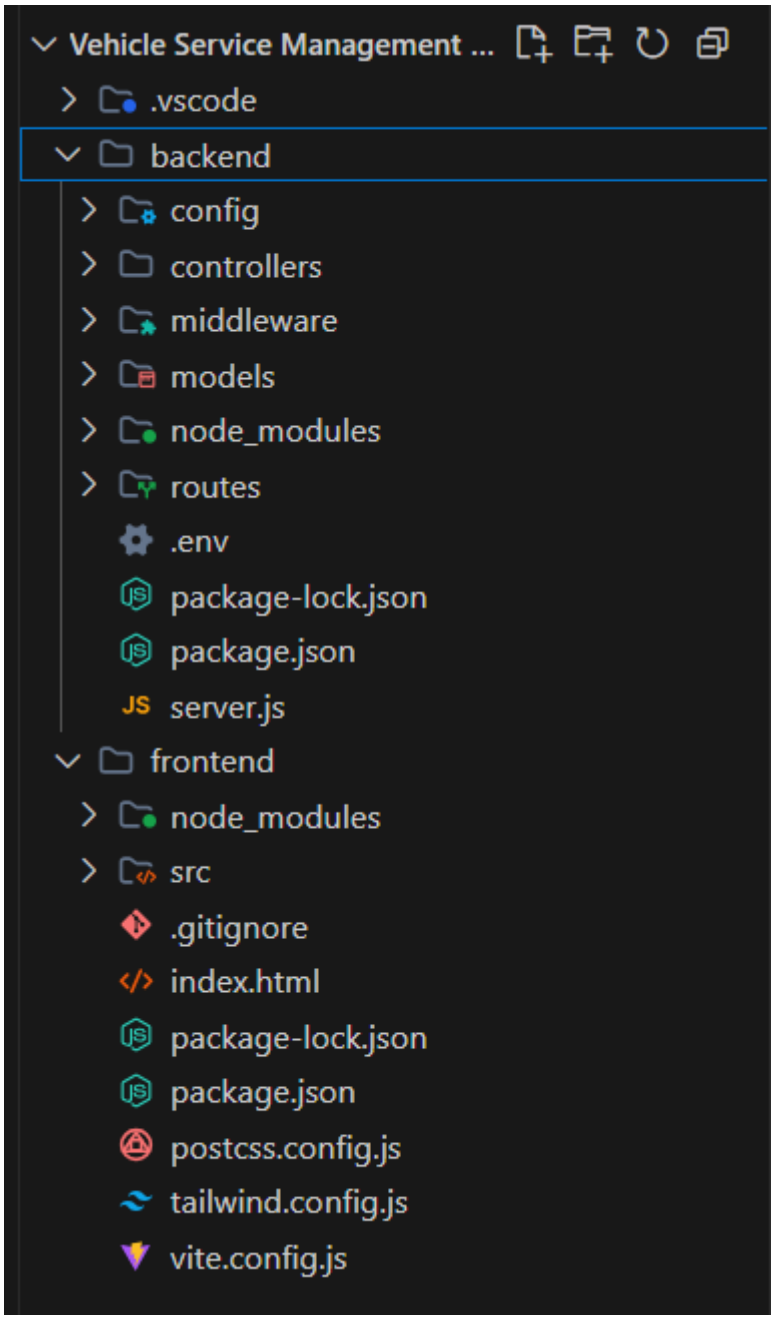
Working of MVC in the System:
When a user interacts with the application (for example, booking a service), the request is sent from the View to the Controller. The Controller processes the request and calls the Model to perform database operations. The Model interacts with the database and returns the required data to the Controller. Finally, the Controller sends the response back to the View, which updates the user interface accordingly.

Advantages of MVC in This Project:

- **Separation of Concerns:** Each layer has a specific responsibility, making the system easier to manage.
- **Scalability:** New features can be added without affecting existing code.
- **Reusability:** Components and logic can be reused across the application.
- **Maintainability:** Code is organized and easier to debug or update.

- **Testing:** Each layer can be tested independently.

Project Setup and Configuration:
The Vehicle Service Management Platform is developed using a full-stack MERN architecture. The project is organized into two main folders: frontend and backend, which separate the client-side and server-side logic.



Creating Project Structure

1. Create a new folder with the project name.
2. Inside the folder, create two directories:

- **frontend** → for React application
- **backend** → for Node.js server

3. Open the project folder in Visual Studio Code.

Frontend Setup (React + Vite):

The frontend is developed using React with Vite for faster development.

- Navigate to frontend folder: **cd frontend**
- Create React app: **npm create vite@latest . -- --template react**
- Install dependencies: **npm install**
- Run the application: **npm run dev**

The frontend includes folders like components, pages, context, services, and utils to organize UI and logic.

Backend Setup (Node.js + Express)

The backend is developed using Node.js and Express.

- Navigate to backend folder: **cd backend**
- Initialize project: **npm init -y**
- Install dependencies: **npm install express mongoose cors dotenv jsonwebtoken bcrypt**
- Create main file: server.js
- Create folders:
- models/
- controllers/
- routes/
- middleware/
- config/

Database Configuration

- MongoDB is used as the database
- Mongoose is used for schema definition
- Database connection is handled in a config file

Running the Application

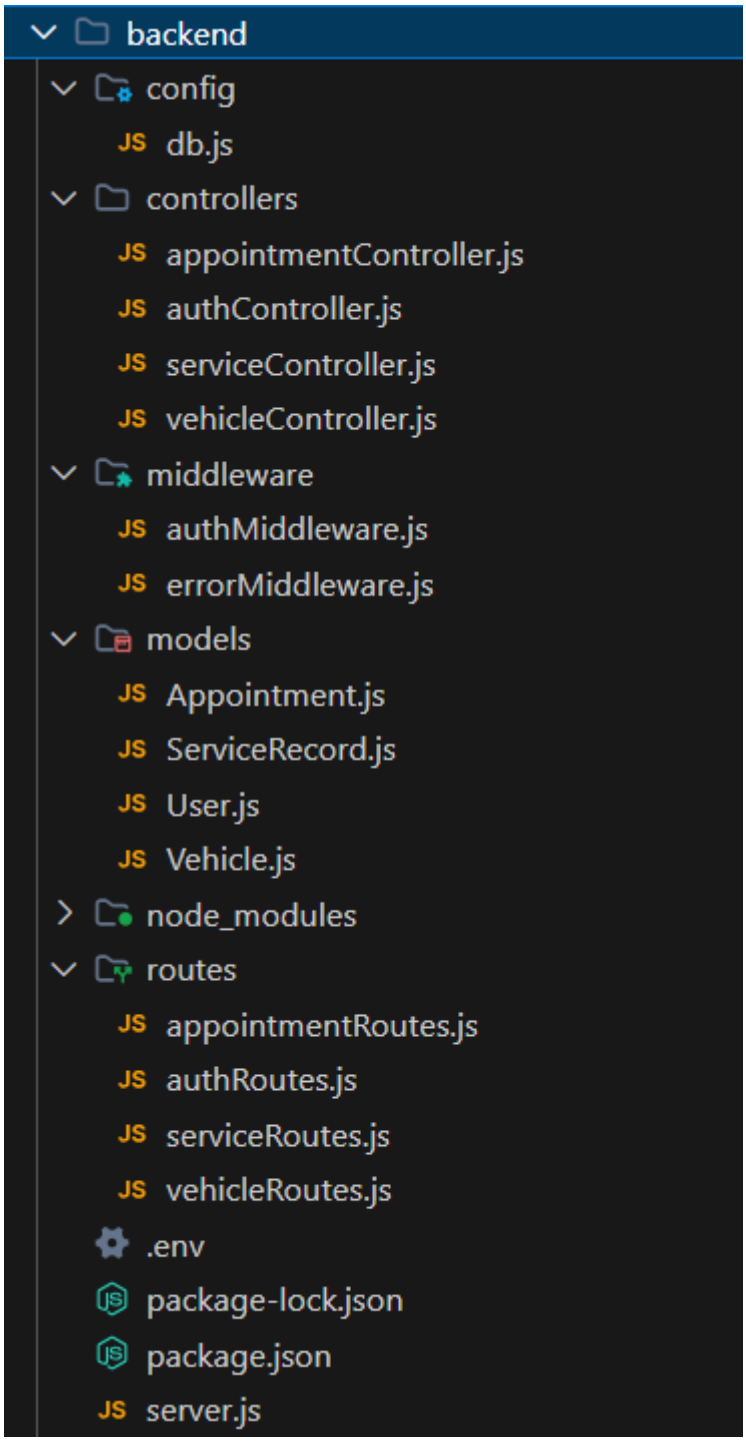
- Start backend: **node server.js**
- Start frontend: **npm run dev**

The project setup ensures a clear separation between frontend and backend, making the system modular, scalable, and easy to maintain.

BACKEND DEVELOPMENT:

Backend Structure:

The backend of the Vehicle Service Management Platform is developed using Node.js and Express.js. It follows a modular structure to separate concerns such as routing, business logic, database models, and middleware. This structure improves maintainability, scalability, and code organization.



Controllers:

- **authController.js** → Handles user authentication such as registration, login, and JWT token generation.

- **vehicleController.js** → Manages all vehicle-related operations including adding, updating, viewing, and deleting vehicles.

- **appointmentController.js** → Handles appointment booking, cancellation, status updates, and technician assignment.

- **serviceController.js** → Manages service-related operations and handles service records created after completion.

Config:

- **db.js** → Contains database connection logic using MongoDB and Mongoose. It establishes a connection between the backend server and the database.

Middleware:

- **authMiddleware.js** → Handles authentication and authorization using JWT. It protects routes by verifying user tokens.

- **errorMiddleware.js** → Handles application errors and sends proper error responses to the client.

Models:

- **User.js** → Defines schema for users including roles (Customer, Admin, Technician), login credentials, and profile details.

- **Vehicle.js** → Defines schema for vehicles including model, license plate, and vehicle type.

- **Appointment.js** → Stores booking details such as vehicle, customer, service type, assigned technician, date, time, and status.

- **ServiceRecord.js** → Stores service history including repairs performed, parts replaced, and technician notes.

Routes:

- **authRoutes.js** → Defines API endpoints for authentication (login, register).

- **vehicleRoutes.js** → Defines API routes for managing vehicles.

- **appointmentRoutes.js** → Defines API routes for booking, canceling, and managing appointments.

- **serviceRoutes.js** → Defines API routes for handling service records and history.

Main Server File:

- **server.js** → Acts as the entry point of the backend application. It initializes the Express app, connects to the database, applies middleware, and sets up all routes.

Environment Configuration

- **.env** → Stores sensitive data such as database URI, JWT secret key, and port number.

DATABASE DEVELOPMENT:

1. **Configure MongoDB:** The Vehicle Service Management Platform uses MongoDB as the database to store application data such as users, vehicles, appointments, and service records. **Mongoose** is used as an Object Data Modeling (ODM) library to define schemas and interact with the database.
- Install Mongoose: npm install mongoose
- MongoDB is connected using a connection string stored in the .env file.

2. **Create Database Connection:** Before performing any operations, the backend must establish a connection with MongoDB. This is handled using a configuration file.
   The database connection is implemented in:
- **config/db.js** → Responsible for connecting the application to MongoDB using Mongoose.

The connection ensures that the server communicates with the database securely and efficiently.

backend > config > JS db.js > ...

```
const mongoose = require('mongoose');

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGO_URI);
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error(`Error: ${error.message}`);
        process.exit(1);
    }
};

module.exports = connectDB;

```

3. **Create Schemas & Models:** Schemas define the structure of data stored in MongoDB collections. In this project, Mongoose schemas are used to model different entities.

    a. User Model (User.js):

Defines user details such as name, email, password, and role (Customer, Admin, Technician). It is used for authentication and role-based access control.

```
const userSchema = mongoose.Schema({
    name: {
        type: String,
        required: true
    },
    email: {
        type: String,
        required: true,
        unique: true
    },
    password: {
        type: String,
        required: true
    },
    role: {
        type: String,
        enum: ['Customer', 'Technician', 'Admin'],
        default: 'Customer'
    }
}, {
    timestamps: true
});
```

b. **Vehicle Model (Vehicle.js):** Stores vehicle information including model, license plate, and vehicle type. Each vehicle is linked to a specific user.

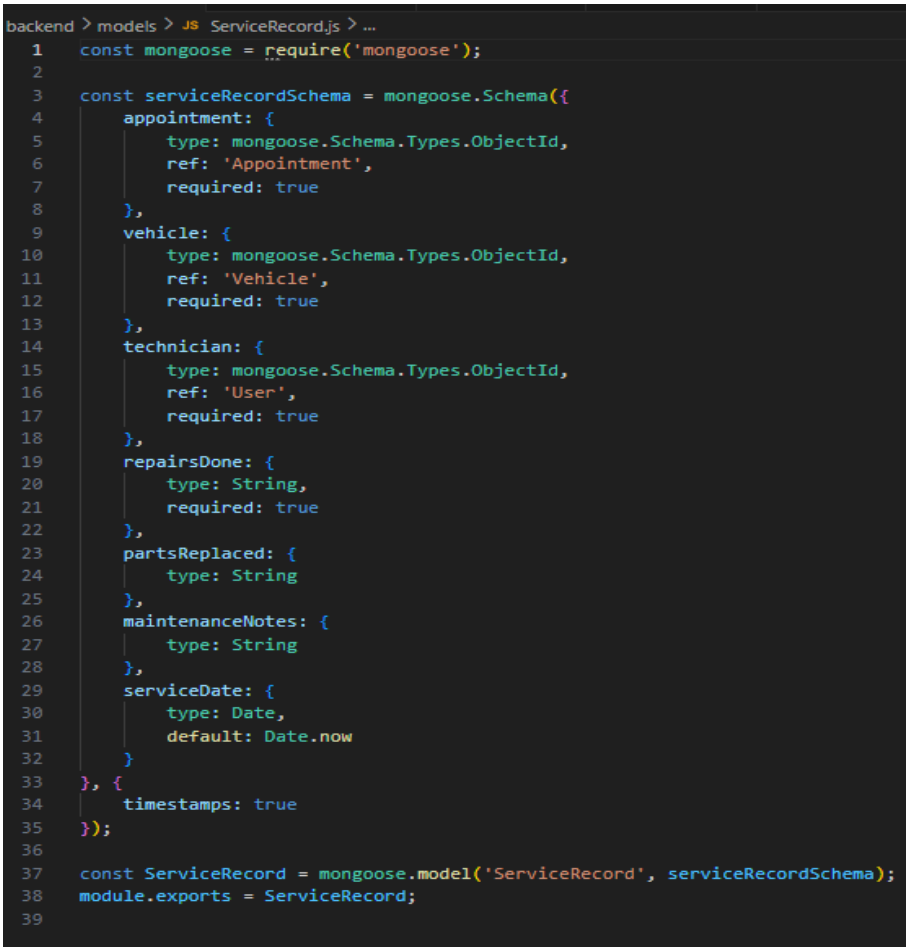
```
const mongoose = require('mongoose');

const vehicleSchema = mongoose.Schema({
    owner: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    model: {
        type: String,
        required: true
    },
    number: {
        type: String,
        required: true,
        unique: true
    },
    type: {
        type: String,
        required: true,
        enum: ['Car', 'Bike', 'Truck', 'Other']
    }
}, {
    timestamps: true
});

const Vehicle = mongoose.model('Vehicle', vehicleSchema);
module.exports = Vehicle;
```

c. **Appointment Model (Appointment.js):** Stores booking details such as vehicle, customer, service type, assigned technician, date, time, and status (Pending, Assigned, In Progress, Completed, Cancelled).

backend > models > JS Appointment.js > ...

```
const mongoose = require('mongoose');

const appointmentSchema = mongoose.Schema({
    customer: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    vehicle: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Vehicle',
        required: true
    },
    serviceType: {
        type: String,
        required: true
    },
    date: {
        type: Date,
        required: true
    },
    time: {
        type: String,
        required: true
    },
    status: {
        type: String,
        enum: ['Pending', 'Assigned', 'In Progress', 'Completed', 'Cancelled'],
        default: 'Pending'
    },
    technician: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User'
    }
}, {
    timestamps: true
});

const Appointment = mongoose.model('Appointment', appointmentSchema);
module.exports = Appointment;
```

d. **Service Record Model (ServiceRecord.js):** Stores service history including repairs performed, parts replaced, and technician notes. It is created after a service is completed.



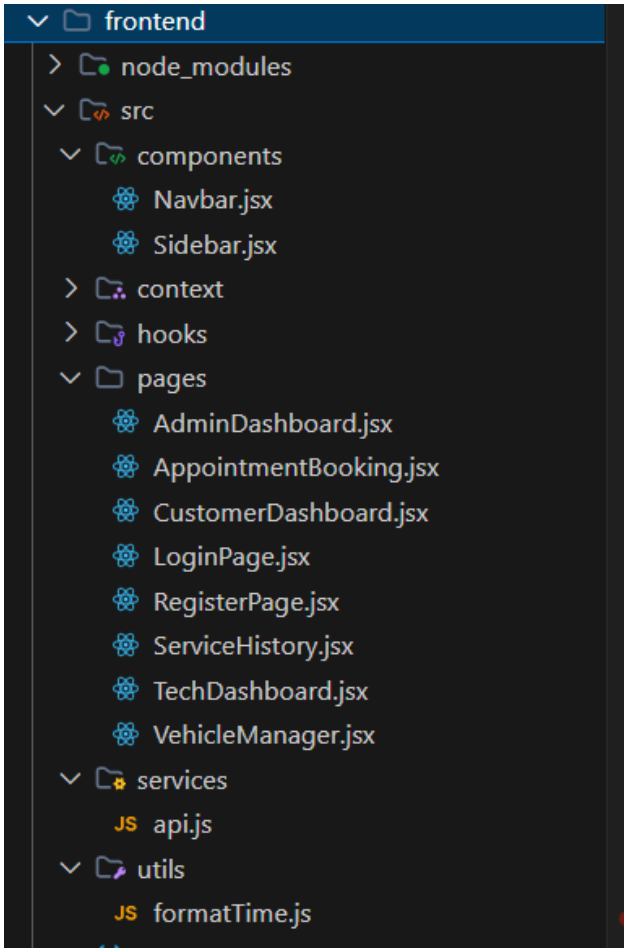
```
backend > models > JS ServiceRecord.js > ...
const mongoose = require('mongoose');

const serviceRecordSchema = mongoose.Schema({
    appointment: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Appointment',
        required: true
    },
    vehicle: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'Vehicle',
        required: true
    },
    technician: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    repairsDone: {
        type: String,
        required: true
    },
    partsReplaced: {
        type: String
    },
    maintenanceNotes: {
        type: String
    },
    serviceDate: {
        type: Date,
        default: Date.now
    }
}, {
    timestamps: true
});

const ServiceRecord = mongoose.model('ServiceRecord', serviceRecordSchema);
module.exports = ServiceRecord;

```

4. Database Relationships:

- One **User** can own multiple **Vehicles**
- One **Vehicle** can have multiple **Appointments**
- Each **Appointment** is linked to one **Service**
- Each **Appointment** may be assigned to a **Technician**
- Each **Appointment** generates one **Service Record**

FRONT-END DEVELOPMENT:

Frontend Structure: The frontend of the Vehicle Service Management Platform is developed using React.js with Vite for faster development and Tailwind CSS for styling. The frontend is responsible for user interaction, displaying data, and communicating with the backend through API calls.
The project follows a modular structure where components, pages, services, and utilities are separated for better maintainability and scalability.



Pages:

- **LoginPage.jsx** → Allows users to log in securely using their credentials.
- **RegisterPage.jsx** → Provides user registration with role selection (Customer, Admin, Technician).
- **CustomerDashboard.jsx** → Displays user-specific information such as vehicles, bookings, and service history.
- **AdminDashboard.jsx** → Provides system overview, appointment management, and technician assignment.

- **TechDashboard.jsx** → Displays tasks assigned to technicians and allows status updates.
- **AppointmentBooking.jsx** → Allows customers to book services by selecting vehicle, date, and time.
- **VehicleManager.jsx** → Enables users to add and manage their vehicles.
- **ServiceHistory.jsx** → Displays past service records and maintenance details.

Components:

- **Navbar.jsx** → Displays user profile, notifications, and logout options.
- **Sidebar.jsx** → Provides navigation between different pages based on user roles.

Context API:

- **AuthContext.jsx** → Manages authentication state across the application, including login status and user role.

Services:

- **api.js** → Handles API calls to the backend using Axios. It manages communication between frontend and backend.

Utilities:

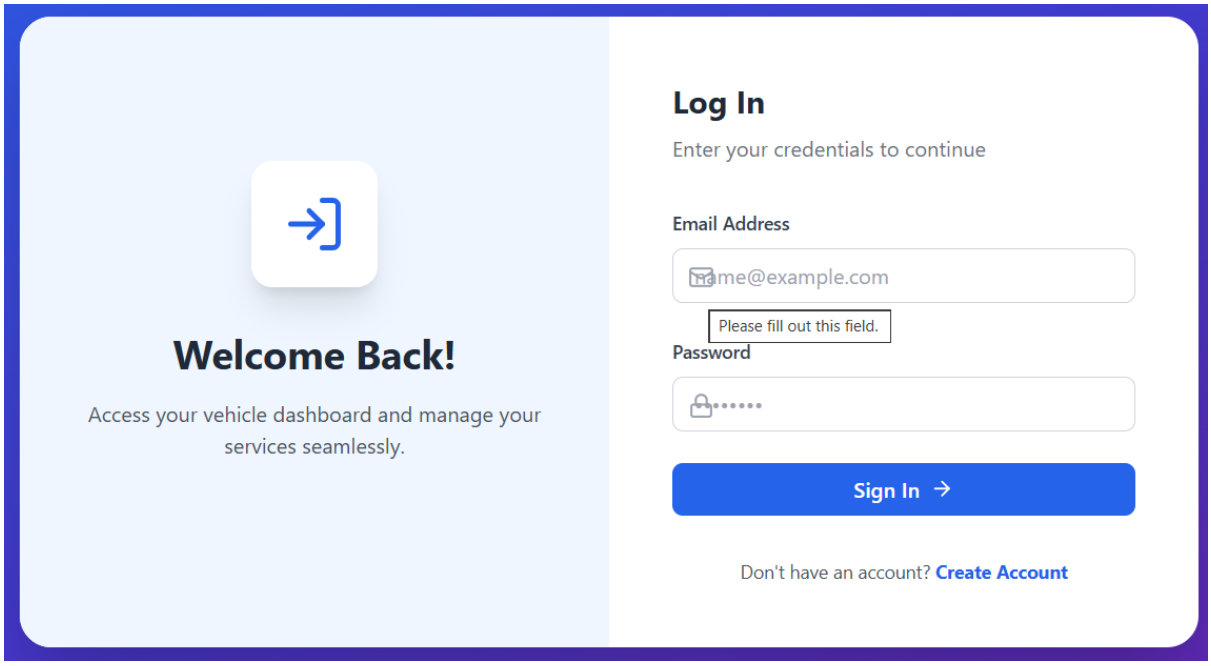
- **formatTime.js** → Contains helper functions for formatting date and time values.

Routing:

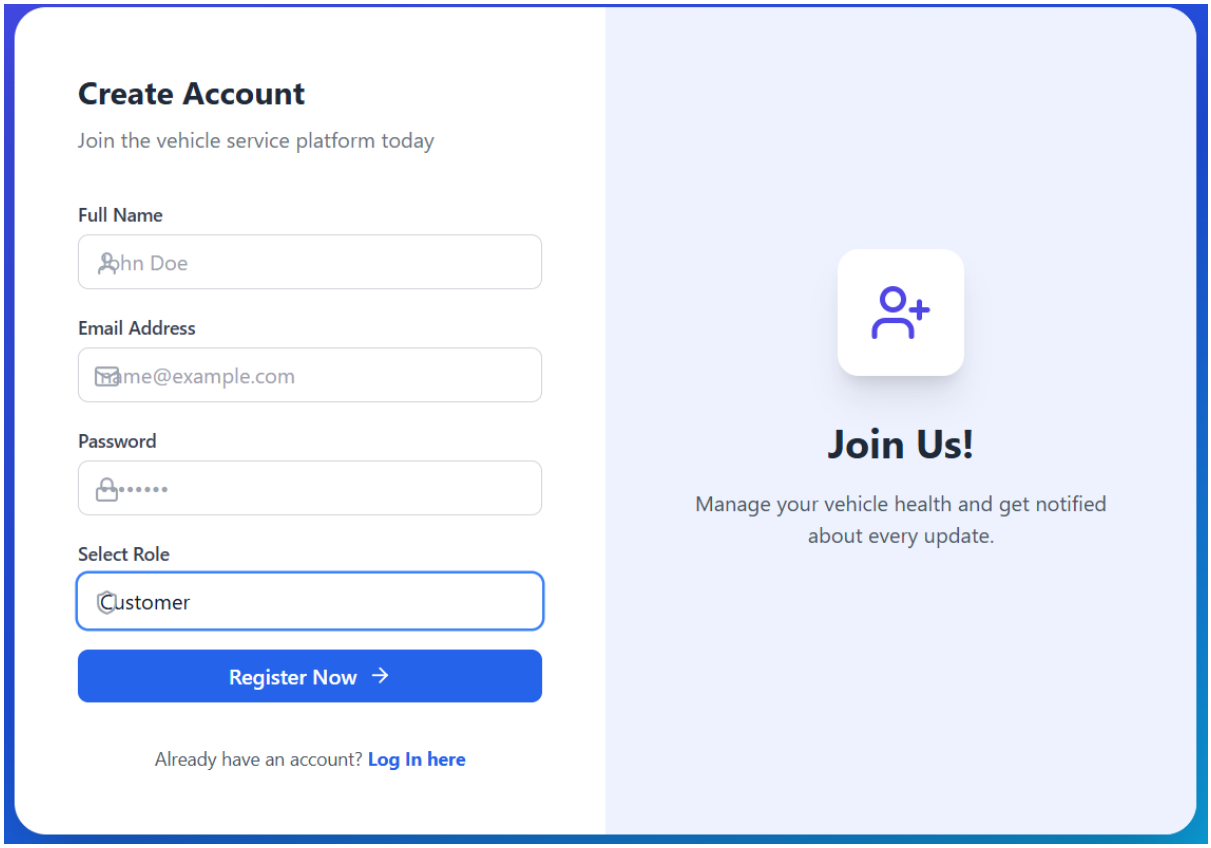
The application uses **React Router DOM** for navigation between pages. It enables client-side routing without reloading the page and supports protected routes based on user roles.

Output ScreenShots:

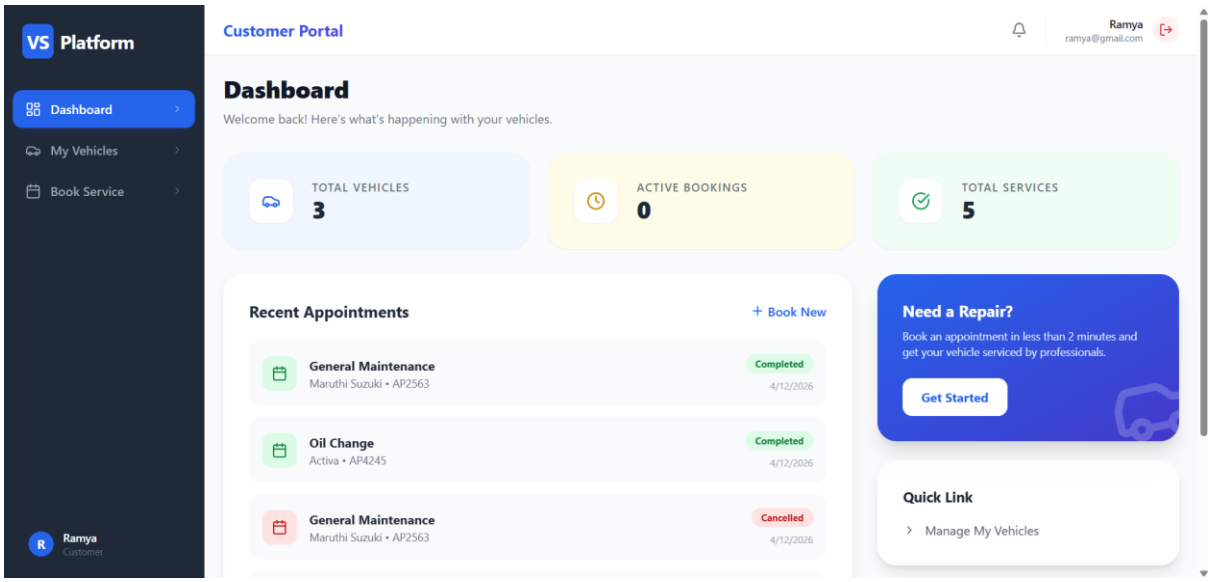
Login Page:



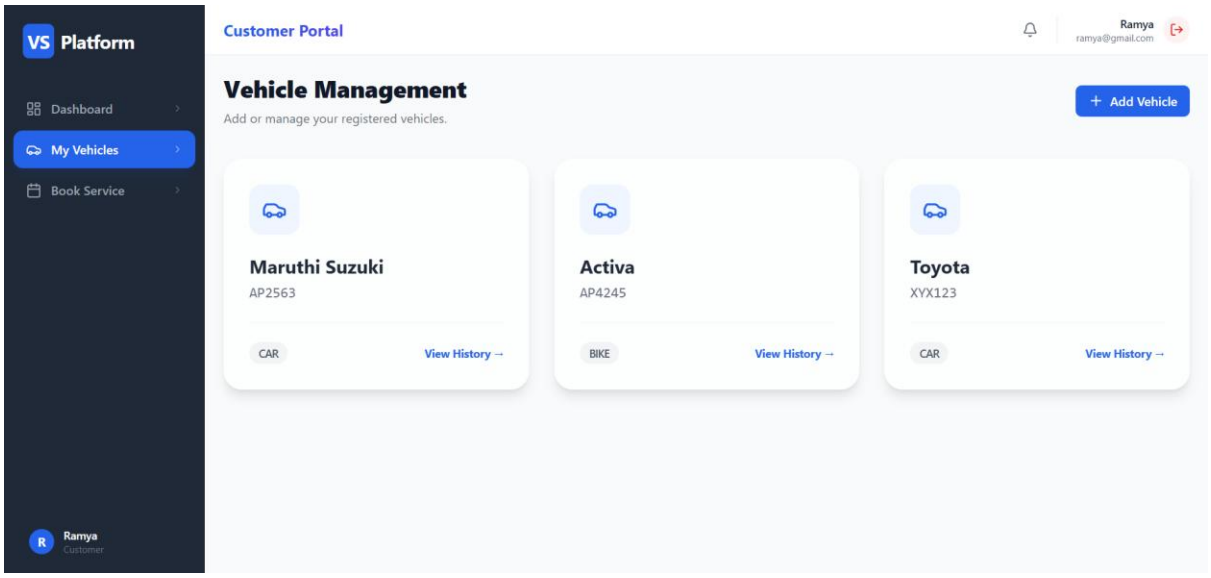
Register Page (Role: Customer/Admin/Technician):



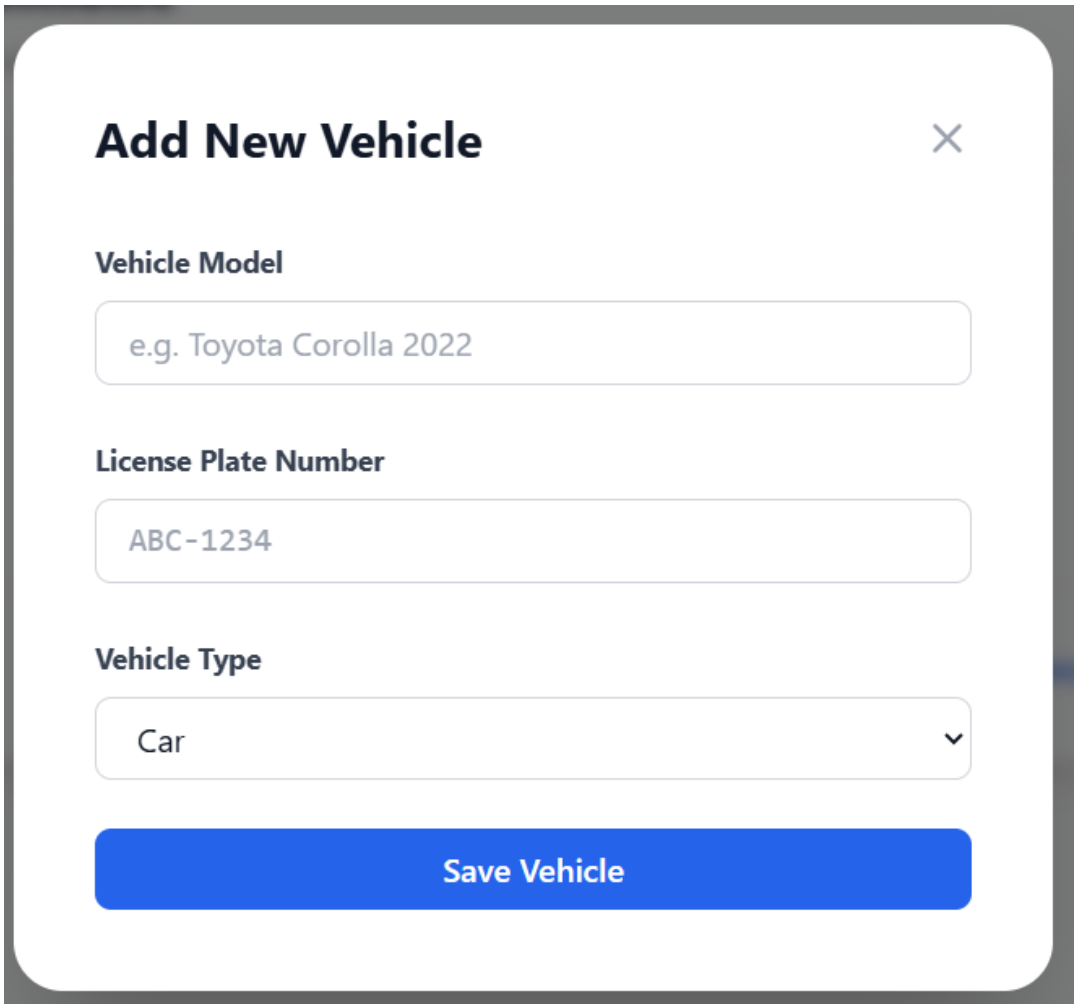
Customer Dashboard:



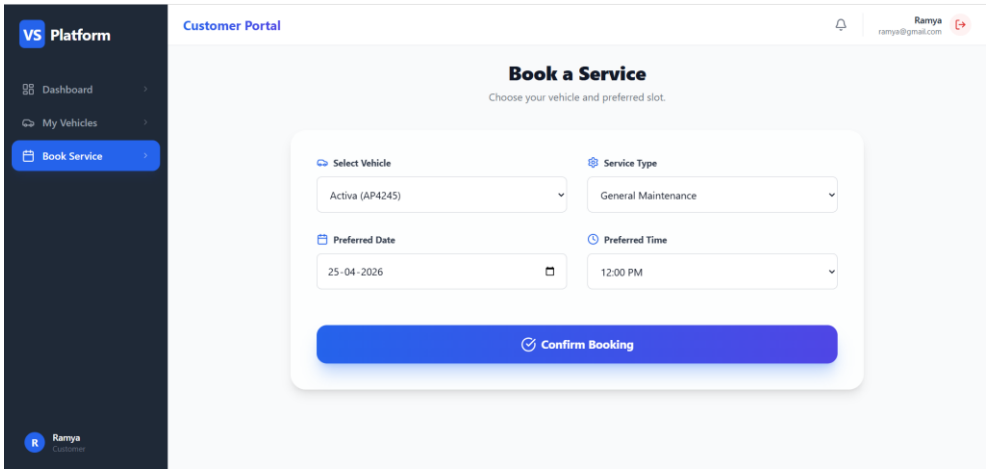
Customer Vehicles:



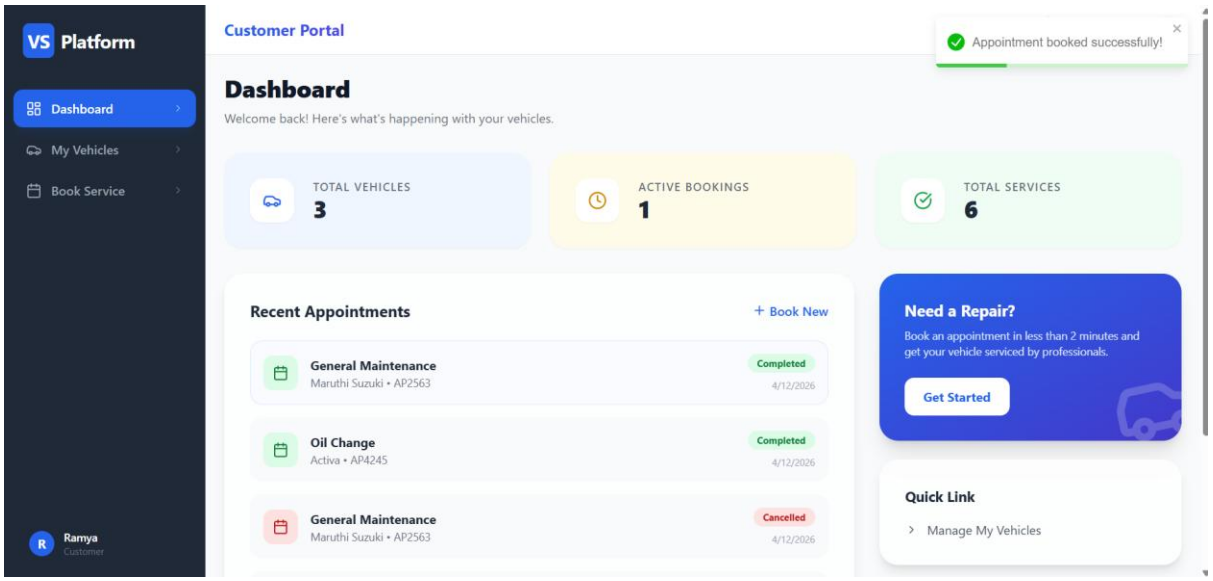
Add new vehicle:



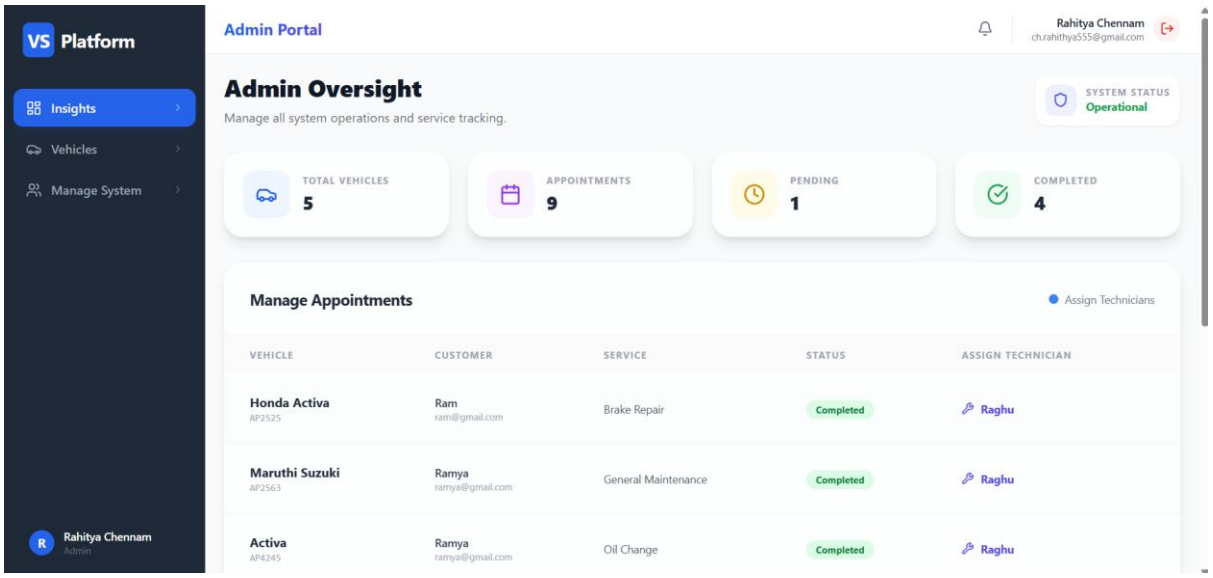
Booking a service:



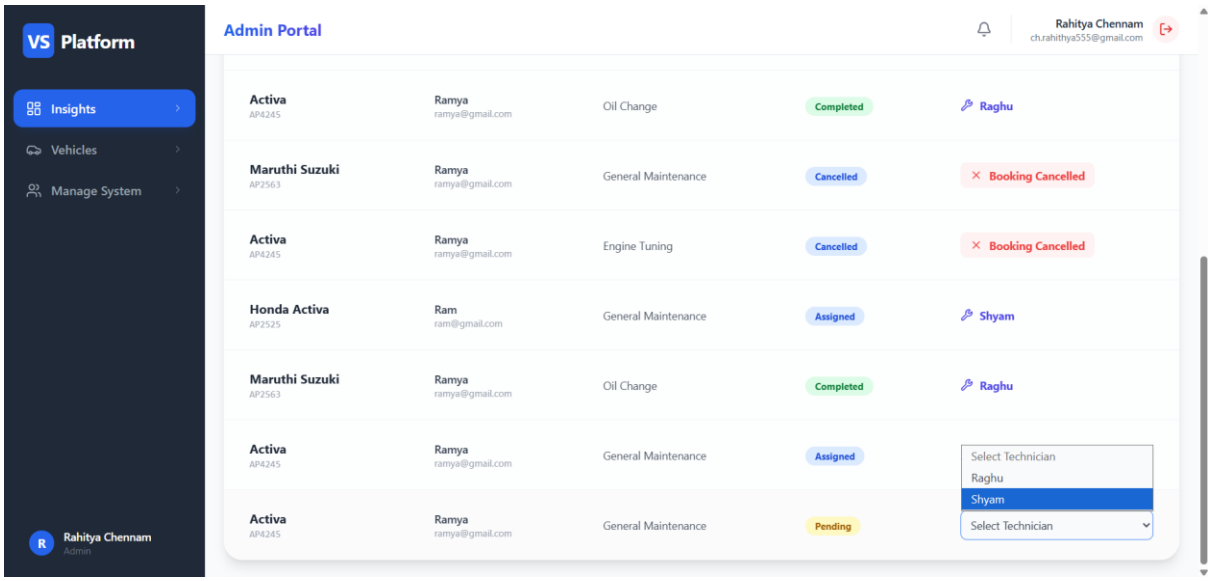
Appointment Booked:



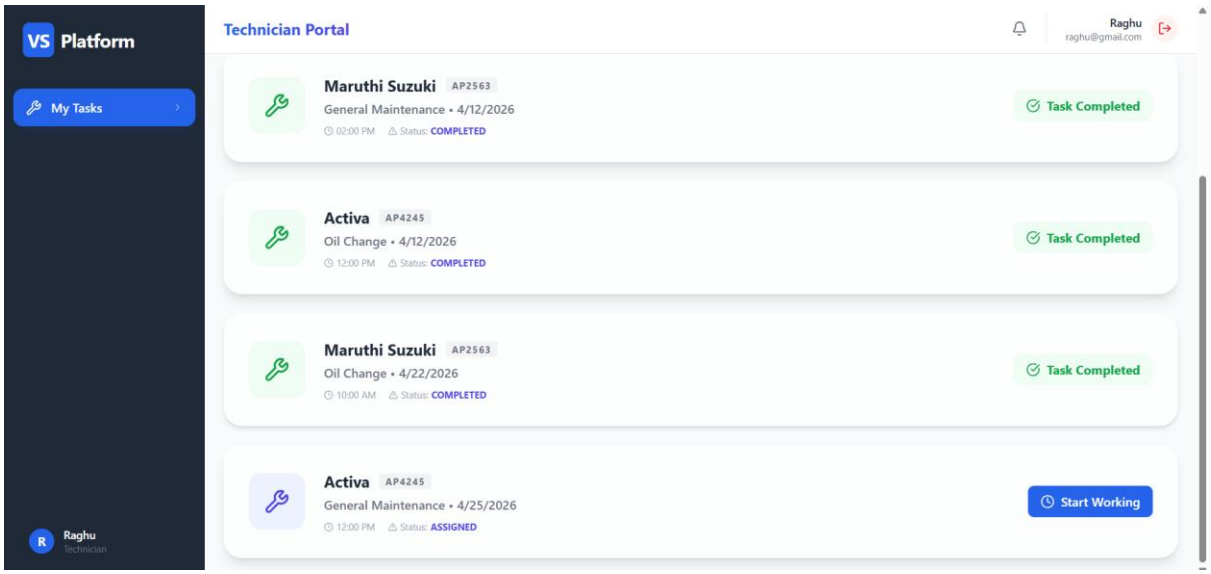
Admin Portal:



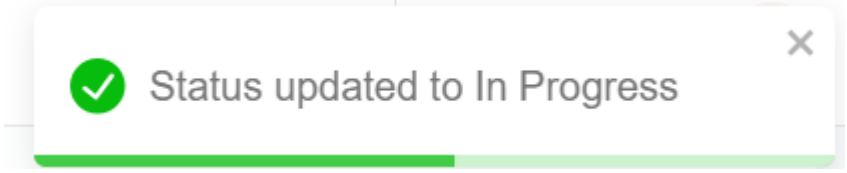
Assigning Technician:



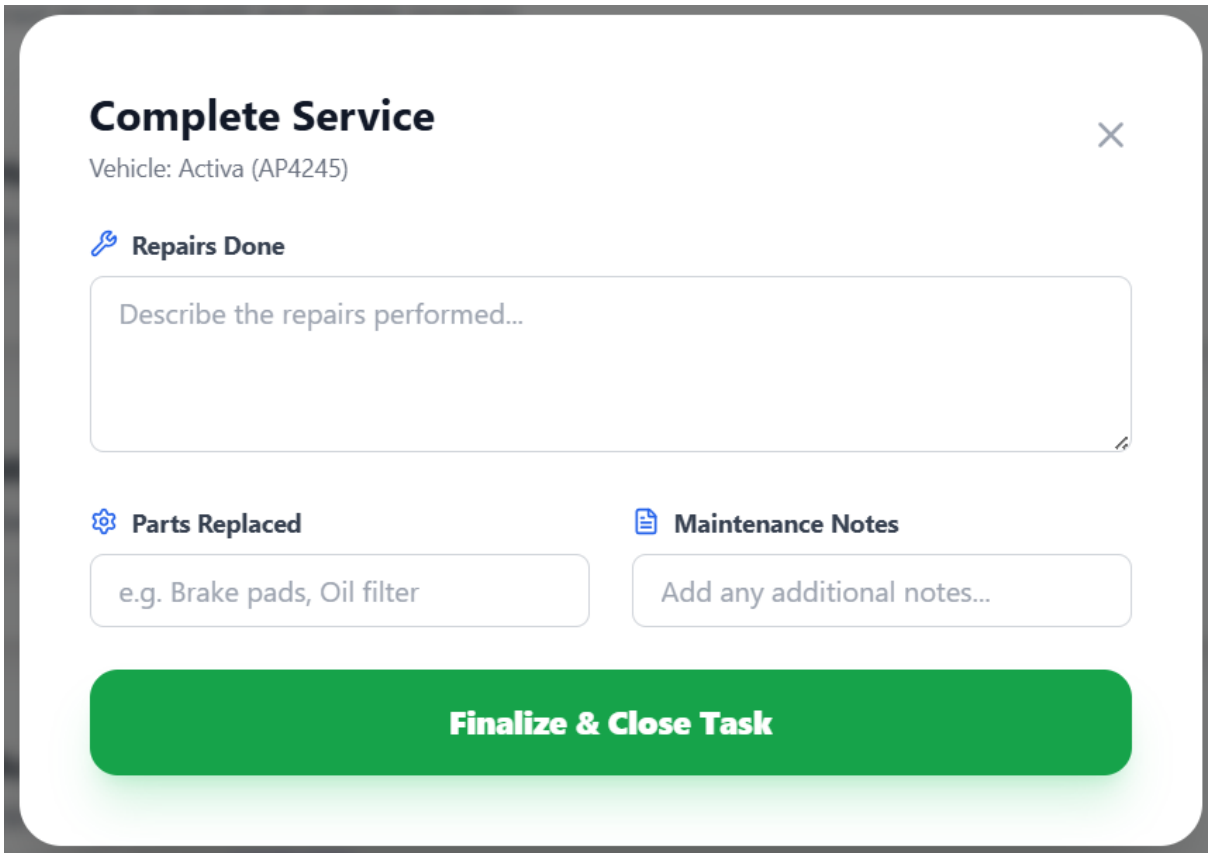
Work assigned to Technician Raghu:

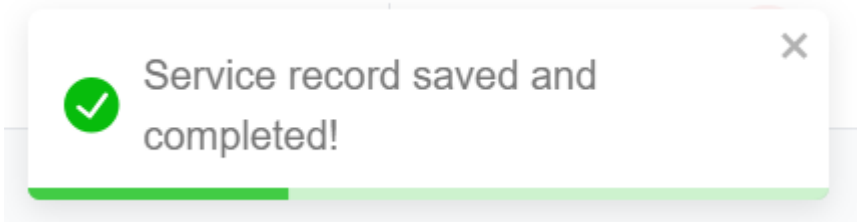


When we click on “Start Working”:

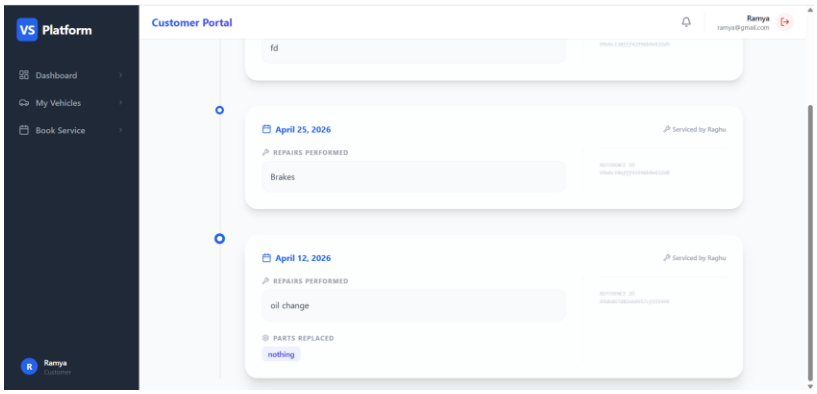


Technician feedback:

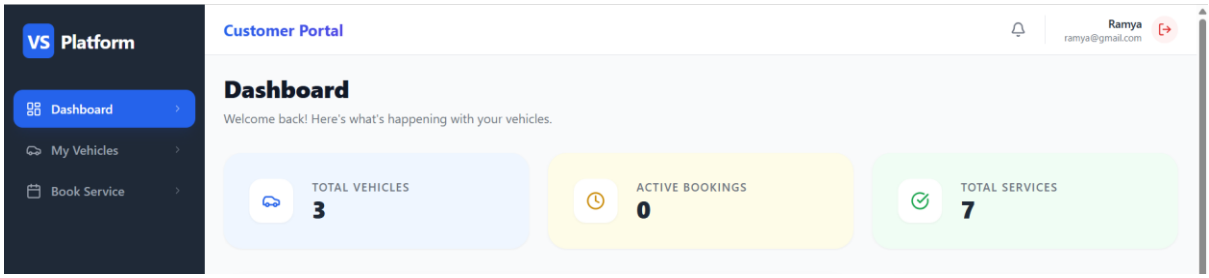




Technician feedback received to customer:



As the service is done there will be no active bookings:



Code Explanation Video Link:
https://drive.google.com/file/d/1OeX866SYgyI5S2G1hhN7IKZkiwsQBjNP/view?usp=sharing

Project Demonstration Video Link:
https://drive.google.com/file/d/1cWXiC4l-Lo63Or2pQrowc7Y4xNtp4RqB/view?usp=sharing

Github Repository Link: https://github.com/Rahitya28/Vehicle-Service-Management.git